

DATA 442/610: Deep Learning and Neural Networks

Assignment 2: CNNs and Image Data

Assigned: Oct 5, 2026; Due: Oct 23, 2026

Instructions

This assignment will be graded out of 100 points. It contains three problems. You are welcome to solve as many problems as you wish. Your score will be calculated based on the problems you turn in, up to a maximum of 100 points. **Any bonus questions will be scored and added on top of the 100-point grade total.**

- **Example:** You try Problem 1 and Problem 2, scoring 40 points on each. You do not do Problem 3. Your grade will be 80/100.
- **Example:** You try Problem 1 and Problem 2, scoring 40 points on each. You do part of Problem 3 and score 10 points. Your grade will be 90/100
- **Example:** You try all three problems, scoring 45 points on Problem 1, 35 points on Problem 2, and 30 points on Problem 3. Your grade will be 100/100

Please submit each problem you attempt as a separate Jupyter notebook with labeled headings/subheadings for each question. This assignment will be graded based on demonstrated understanding of the core concepts. You do not need to achieve the best possible neural network accuracy. High marks will come from thoughtful documentation and explanation of *what* you are attempting, *how* you expect it to perform, and *why* you believe it will behave like it does. **Please comment your code.**

This assignment may require working out some intermediate mathematical steps. You are encouraged to show your work on any derivations underlying your code, and include this alongside your submission. You may submit any mathematical derivations as typeset equations in your Jupyter Notebooks. If you prefer not to type out that many equations, you may include a scanned PDF of handwritten calculations.

Problem 1 (50 points) - Fashion-MNIST classification

In this problem you will develop a CNN to classify the Fashion MNIST dataset. This dataset has a convenient ready-to-use implementation in PyTorch: `torchvision.datasets.FashionMNIST`. A minimum accuracy of 80% is expected, although you are welcome to tune your model to achieve higher performance.

Part 1: Data Preprocessing and Augmentation (10 points)

Begin by loading the Fashion-MNIST dataset in `torchvision`. Use the following snippet to get started. Check the size and shape of the training and testing datasets.

```
import numpy as np
import torch
import torchvision
from torchvision.transforms.v2 import functional as F

# Create datasets for training & testing, download if necessary
train_dataset = torchvision.datasets.FashionMNIST(root='~/scr10/', train=True,
                                                  download=True)
test_dataset = torchvision.datasets.FashionMNIST(root='~/scr10/', train=False,
                                                  download=True)

# Make the test dataset a bit harder by rotating +/- 30 degrees
np.random.seed(42)
for ii in range(len(test_dataset)):
    test_dataset.data[ii] = F.rotate(test_dataset.data[ii, None],
                                     angle=np.random.uniform(-30, 30)).squeeze()
```

Question 1: Using PCA or another dimensionality reduction method, project the training data onto a two-dimensional feature space and plot the distribution of each image class. *Without re-fitting the PCA estimator*, plot the distribution of the test data using the on a separate axis. Comment on the relative distributions of the training and test datasets.

Question 2: Using a `scikit-learn` model of your choice, fit using the training dataset and report the test accuracy.

Question 3: Create a `torchvision` transform that applies a random rotation to the training images within a range of your choice. Create a plot that shows original training set alongside rotated images, after applying the transform. Do not apply this transform to the testing dataset.

Question 4: Preprocess the data by normalizing the input pixel values such as by rescaling pixels, or standardizing to a baseline mean and standard deviation. You may consider other preprocessing steps, such as image resizing, cropping, or others. Describe your final preprocessing pipeline and how you expect it to impact the training process.

Part 2: CNN Implementation (25 points)

Implement and train a CNN to classify the Fashion MNIST dataset. Be sure to split your dataset into a training and validation set (we recommend an 80-20 split) to monitor training performance. You are free to experiment with your architecture, but it should at a minimum include:

- at least one convolutional layer

- pooling layers (e.g. max pooling, average pooling)
- nonlinear activation functions (e.g. ReLU, tanh, sigmoid, or others)
- one or more fully connected layers before the output layer

You may also consider options such as regularization, dropout, normalization layers, skip connections, strided/grouped convolutions, or others (attention modules and vision-transformer blocks are not permitted).

Monitor training and validation loss and aim for a minimum validation accuracy of 80%. You are encouraged to aim for higher accuracy. Beyond architectural tweaks, you may also consider alternative optimizers, learning rate decay/scheduling, early stopping, batch size variation, etc.

Once you have settled on a final model, explain your choice of architecture and training recipe. Comment on experiments or trials you ran prior to your final model, and how specific configurations affected training and validation performance.

Finally, measure and report accuracy on the rotated test dataset. Compare this result to the `scikit-learn` baseline.

Part 3: Performance Analysis (15 points)

PS1 contained a problem on classifying this same dataset using a multi-layer perceptron. Here, you will compare the CNN model above against an MLP baseline and examine how performance scales with each architecture's size.

Starting from your finalized architecture and training recipe from Part 2, conduct additional trials where you vary **either** the depth (number of convolutional layers) or width (number of features within each layer). Train at least 4 total models holding all other hyperparameters constant. For each model, measure the test accuracy and the number of parameters using the code snippet below.

```
num_trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Model has {num_trainable_params} trainable parameters')
```

Next, implement a multi-layer perceptron using PyTorch to classify the Fashion-MNIST dataset. Within reason, aim to give it a similar structure as your CNN from Part 2 (similar number of hidden layers, same activations, or approximately equal parameter count). From this baseline, conduct training trials using your Part 2 training recipe. If you varied depth in your CNN analysis, vary depth in your MLP trials. If you varied width in your CNN analysis, vary width here as well. Train at least 4 total models holding all other hyperparameters constant. For each model, measure the test accuracy and the number of parameters. As a guide, your results should be able to populate a table like Table 1.

For each model, plot the number of trainable parameters against the test accuracy. Comment on how each architecture's performance scales. If one is more efficient than the other, explain why this might be the case.

Trial #	Model Type	# Layers	Layer Width	# Params	Test Accuracy
1	MLP	2	32	...	...%
2	MLP	3	32	...	...%
3	MLP	4	32	...	...%
4	MLP	5	32	...	...%
5	CNN	2	32	...	...%
6	CNN	3	32	...	...%
7	CNN	4	32	...	...%
8	CNN	5	32	...	...%

Table 1: Example results table, for trials varying depth

[Bonus] Part 4: ConvNeXt (10 points)

ConvNeXt (see <https://arxiv.org/abs/2201.03545>) is a fully convolutional neural network architecture that aims to compete with large-scale transformer models (which we will discuss later). The basic ingredients of a ConvNeXt block are

1. Depth-wise, or grouped, convolution with a large kernel (5×5 or 7×7)
2. Normalization, such as `nn.BatchNorm` or `nn.LayerNorm`
3. Expanding convolution ($1 \times \rightarrow 4 \times$) with a 1×1 kernel
4. GELU activation
5. Contracting convolution ($4 \times \rightarrow 1 \times$) with a 1×1 kernel
6. Residual or skip connection (`block_out` $\leftarrow$ `block_in` + `conv_out`)

Using your final CNN architecture from Part 2 as a starting point, replace `Conv2d` + `activation` blocks with ConvNeXt blocks. Repeat your depth or width scaling analysis and plot parameter count vs. test accuracy for your ConvNeXt models. Comment on their relative performance compared to your baseline CNN.

Problem 2 (50 points) - Image Segmentation

In this problem you will develop a CNN model for Image Segmentation using the Oxford IIIT Pet Dataset. You can find documentation for this dataset here: <https://www.robots.ox.ac.uk/~vgg/data/pets/>. **Image Segmentation** aims to partition an image into distinct regions, each belonging to a particular class or object. This dataset contains ground-truth segmentations where each pixel is labeled as either **foreground**, **background**, or **unknown**. The goal is to develop a machine learning model that correctly label as many foreground pixels as possible, while minimizing the number of incorrectly-labeled background pixels.

Part 1: Data Preprocessing (10 points)

Begin by loading the Oxford-IIIT-Pet dataset in `torchvision`. Use the following snippet to get started. Be sure to check the size and shape of the training and testing datasets.

```
import torchvision
from torchvision.transforms import v2

# Create datasets for training & validation, download if necessary
train_val_dataset = torchvision.datasets.OxfordIIITPet(root='~/scr10/', split='trainval', target_types='segmentation', download=True)
test_dataset = torchvision.datasets.OxfordIIITPet(root='~/scr10/', split='test', target_types='segmentation', download=False)
```

Question 1: Visualize at least 4 samples from the dataset. Each sample will contain an image and a corresponding segmentation mask.

The entries of this dataset do not have a standard size. This means that `torch.utils.data.DataLoader` may have trouble collating samples from each batch. To get around this, you will write a custom transformation that standardizes each sample. You may start with this template.

```
from torchvision.transforms.v2.functional import resize, pad, crop
class StandardizeImage:
    def __init__(self, target_size=224, *args, **kwargs):
        self.target_size = target_size

    def __call__(self, x, y):
        """ Transforms x and y from size [..., H, W] to
            [..., target_size, target_size]. Be sure to
            process both x and y in the same way so the
            segmentations match up!
        """
        pass
```

In addition, the segmentation masks are not quite ready for classification. They need to be converted to `torch.long` tensors whose entries give an index corresponding to a particular class (foreground, background, unknown). It is up to you how to implement this. For example, you could write a separate transform, extend the `OxfordIIITPet` Dataset class, or take some other approach.

Question 2: Using your custom standardization transform, visualize at least 4 paired samples from the dataset. Confirm that the resizing, padding, or cropping was applied evenly to both the input image and the target segmentation. Include a colorbar or legend alongside the segmentation mask. Comment on how your choice of image preprocessing might affect the behavior of a CNN.

Question 3: Augment your training dataset using composed image transformations. This may include flips, shifts, rotations, color balance changes or anything else you like. Explain why you chose the specific augmentation options. Visualize at least 4 augmented samples from your dataset.

Part 2: U-Net Implementation (35 points)

In this part, you will implement and train a U-Net model. U-Net is a standard architecture for image segmentation (read more here: <https://arxiv.org/abs/1505.04597>).

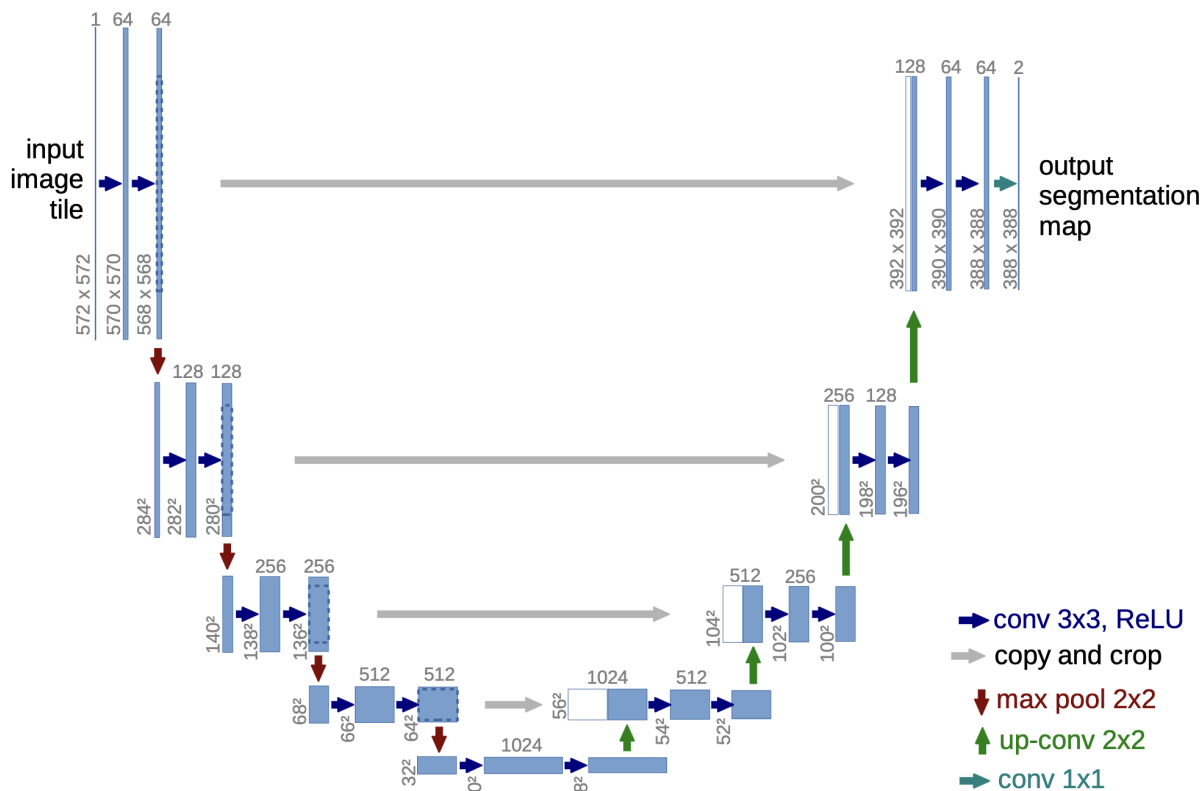


Figure 1: U-Net architecture, see <https://arxiv.org/abs/1505.04597>

A U-Net contains two components: an **encoder** and a **decoder**. The encoder contains a series of modules that apply 2-D convolutions, nonlinear activations, and downsampling (pooling) operations. The decoder contains modules that apply 2-D convolutions, nonlinear activations, and upsampling operations. The upsampling is done via a transposed convolution (see `torch.nn.ConvTranspose2d`).

Crucially, there are “skip connections” between the encoder and decoder block at each level. This means that the outputs of each encoder block (grey horizontal arrows in Fig. 1) are concatenated with the outputs of each upsampling operation (green vertical arrows in Fig. 1).

In this part, you are not required to use the exact specification from the original paper or Fig. 1. You may vary any hyperparameters as you see fit. The objective here is to implement the overall macro-architecture: encoder, decoder, and skip connections.

Question 1: Create an `EncoderBlock` class that extends `torch.nn.Module`. Each block should

contain at least one convolution and at least one activation function.

Question 2: Create an `Encoder` class that extends `torch.nn.Module`. The encoder should contain $N \geq 3$ levels containing `EncoderBlock` modules. The first $N - 1$ `EncoderBlocks` should be followed by a downsampling or pooling operation of your choice. The `Encoder` should return the outputs of each `EncoderBlock` as a list.

Question 3: Create a `DecoderBlock` class that extends `torch.nn.Module`. Each block should contain at least one convolution and at least one activation function.

Question 4: Create a `Decoder` class that extends `torch.nn.Module`. The decoder should contain the same number of levels N as the encoder in Question 2. Its `forward` function should accept the outputs at each level of the encoder. Each decoder level should upsample the outputs of the previous level, concatenate them with the corresponding encoder outputs, and then apply a `DecoderBlock`. Use Fig. 1 as a guide.

Question 5: Comment your code with the expected tensor sizes/shapes at each level. Check that your model's forward function works on a sample element from the dataset.

Question 6: Train your model using `CrossEntropyLoss` to classify each pixel. You may use any optimizer and other strategies as you see fit such as label smoothing, class weighting, etc. In addition to training/validation loss, record and plot the **intersection over union** for the foreground class (fg) at each epoch:

$$\text{IOU}(\hat{y}, y_{\text{NN}}) = \frac{\text{Area}[(\hat{y} = \text{fg}) \cap (y_{\text{NN}} = \text{fg})]}{\text{Area}[(\hat{y} = \text{fg}) \cup (y_{\text{NN}} = \text{fg})]}$$

Aim for a validation IOU of at least 0.5. You are encouraged to tune your model to reach higher performance.

Part 3: Performance Analysis (5 points)

Question 1: Compute the average intersection over union using the test split of the Oxford-IIIT-Pets test dataset. Compare this against a baseline that you choose. Explain why you chose that particular baseline. How does your U-Net model compare?

Question 2: Visualize at least 4 samples from your test dataset. Include the input image, the target segmentation mask, the model prediction, and an error map. The error map should clearly identify where the model misclassifies each pixel. Comment on the performance of the model, and propose some strategies to further improve its performance.

Problem 3 (50 points) - Flocking

Collections of individuals (think birds, fish, or people) often perform coordinated large-scale actions (flocking, schooling, crowding). These behaviors may be organized in a top-down manner, like a general ordering troops on a battlefield, or may emerge from individuals acting autonomously.

The Vicsek model is a theoretical model of agent-based flocking. It defines a collection of agents, each of which moves about a region with constant speed. The agents change direction by measuring the average of their nearby neighbors. In this model, there is a transition from disordered motion to flocking that depends on two properties:

- **Density**, or how many neighbors each agent has
- **Environment noise**, or how often agents make “mistakes” when observing their neighbors

For this problem, you will develop CNN models to perform classification and regression tasks using data from flocking simulations.

Part 1: Data Preprocessing [5 points]

The datasets for this problem are available here (you must be logged into Google Drive using your W&M credentials). You can unzip the file using: `tar xzvf flocking_dataset.tar.gz`. Each dataset contains several folders, each with 100 frames from a particular simulation run. The folders contain the frames in two formats. The first is a PNG image showing each agent’s position and velocity as an arrow. The second is a `.npy` file storing a 2-channel grid. The first channel indicates whether an agent is present at the grid site and the second channel indicates the direction (measured as an angle in radians) that the agent is facing. The two representations are equivalent.

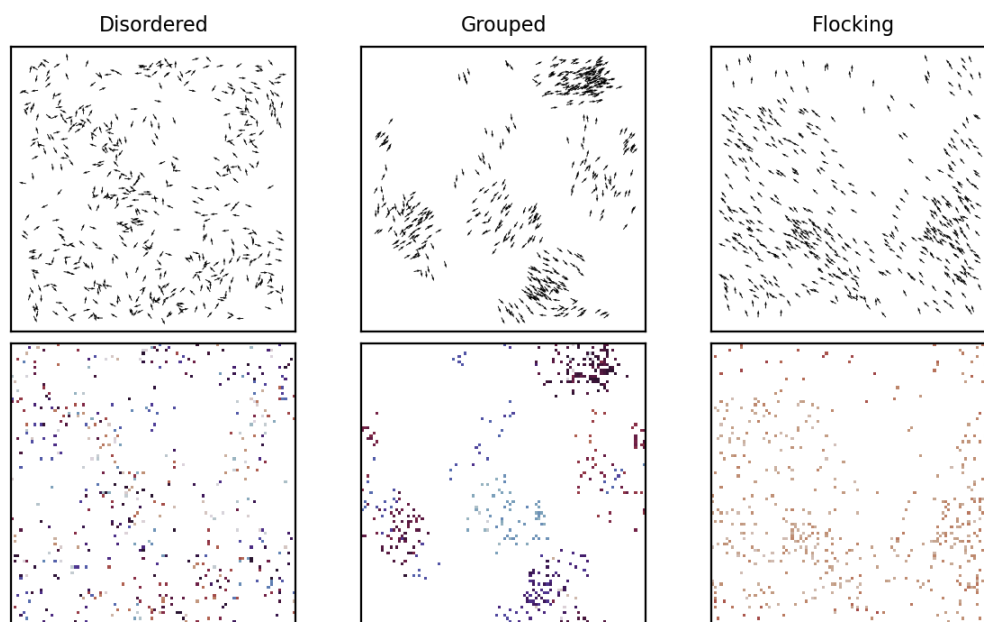


Figure 2: Simulations exhibit three phases. (*top row*) Image data (*bottom row*) Grid data, color indicates the direction each agent faces

The labels for each dataset are stored CSV files, with columns for each simulation folder, frame number, and label (disordered, grouped, flocking).

Question 1: Implement a class that extends `torch.utils.data.Dataset` and provides PNG images and labels as (x, y) pairs.

Question 2: Create another Dataset that loads the `.npy` files and labels as (x, y) pairs.

Question 3: Plot samples of each dataset together for each phase. Use Figure 2 as a guide.

Part 2: Phase classification [20 points]

The data have three labels: disordered, grouped, and flocking. In the disordered phase, the agents do not coordinate and appear to move independently of each other. In the flocking phase, the agents move together along a preferred direction. In the "grouped" phase, the agents may assemble into clusters but lack universal agreement. The goal of this part is to develop a CNN model that can classify these phases from simulation data.

Implement a CNN classifier model that contains at least one convolutional layer, pooling operation, activation function, and fully connected layer. You may consider additional options such as regularization, dropout, normalization, skip connections, strided/grouped convolutions, or others (attention modules and vision transformer blocks are not permitted).

Question 1: Train your CNN classifier using the PNG image dataset. You may adjust your training recipe using different image augmentation strategies, optimizers, learning rate schedules, regularization methods etc. Comment on any experiments or trials you ran, and explain the choices in your final training recipe. Record and plot loss and accuracy on your training and validation sets.

Question 2: Measure and report accuracy on the test PNG dataset.

Question 3: Using the same architecture from Question 1, train a CNN classifier using the `.npy` dataset. You may adjust the input layer to your network for compatibility and may also modify your preprocessing and augmentation steps. Explain any changes you have made and what effect you expect to occur. Train your model using the training recipe from Part 1. Record and plot loss and accuracy on your training and validation sets.

Question 4: Measure and report accuracy on the test `.npy` dataset. Comment on the relative performance compared to the model trained on PNG data. Propose an explanation for any differences that you observe.

Part 3: Parameter regression [20 points]

The transition between disordered and flocking phases can be studied in terms of a global **order parameter** $v_a \in [0, 1]$. The order parameter is near 0 in the disordered phase and near 1 in the flocking phase. This order parameter is provided for each dataset as an additional column in the `labels.csv` file. The goal of this part is to develop a CNN model that can determine the continuous order parameter from simulation data.

Modify your datasets to return the order parameter v_a instead of the phase label. Likewise, modify your CNN model and training pipeline from Part 2 to perform regression. Explain any changes that you have made to your architecture and training procedure.

Question 1: Train a CNN regressor model using the PNG image data. Record and plot the loss on your training and validation sets.

Question 2: Using the test PNG dataset, measure and report the Coefficient of Determination (R^2) between ground truth values of v_a and regression model predictions.

Question 3: Train a CNN regressor model using the .npy image data. Record and plot the loss on your training and validation sets.

Question 4: Using the test .npy dataset, measure and report the Coefficient of Determination (R^2) between ground truth values of v_a and regression model predictions. Comment on the relative performance compared to the model trained on PNG data. Propose an explanation for any differences that you observe.

Part 4: Classification revisited [5 points]

The order parameter v_a is an indicator of the system's phase. Thus, an accurate regression model that predicts v_a can be a **backbone** for classification. In this part, you will adapt the regression model from Part 3 into a classifier model.

Freeze the weights of your best regression model (either PNG or .npy) by setting each model parameter's `requires_grad` attribute to `False`. Implement a new neural network that incorporates the frozen regression model as part of a classification pipeline.

Question 1: Train your model to perform classification, and track training/validation metrics. Compare the convergence of this classifier to the one you trained in Part 2.

Question 2: Measure and report accuracy on the test set. Comment on the relative performance compared to the model from Part 2. Propose an explanation for any differences that you observe.